Thread Pools[Executor Framework]

11 April 2026 18:10

Creating a new thread for every job may create performance and memory problem, to overcome this, we should go for Thread Pool. Thread Pool is a pool of already created threads ready to do our job.

In other words, we can say that: A Thread Pool is a collection of pre-created worker threads that are reused to execute multiple task.

Instead of:

- Creating a new thread for every task is costly.
- We use a fixed set of threads and assign tasks to them is efficient.

Java 1.5 version introduces Thread Pool framework to implement Thread Pools. Thread Pool framework also known as Executor framework. We can create a Thread Pool as follows:

```
ExecutorService service = Executors.newFixedThreadPool(3);
```

We can submit a runnable job by using submit() method.

```
service.submit(job);
```

We can shut down executor service, by using the shutdown() method.

```
service.shutdown();
```

```
package MyThreads.MultithreadingEnhancement;
public class PrintJob implements Runnable{
    private String name;
    public PrintJob(String name){
        this.name = name;
    }
    @Override
    public void run() {
        System.out.println(name + " ...Job started by Thread: "+
            Thread.currentThread().getName());

        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println(name + " ...Job completed by Thread: "+
            Thread.currentThread().getName());
    }
}
```

```
package MyThreads.MultithreadingEnhancement;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
public class ExecutorDemo {
```

```
    public static void main(String[] args) {
        PrintJob[] jobs = {
            new PrintJob("Shiv"),
            new PrintJob("Parvati"),
            new PrintJob("Ram"),
            new PrintJob("Sita"),
            new PrintJob("Krishna")
        };
        ExecutorService service = Executors.newFixedThreadPool(3);
        for (PrintJob job : jobs) {
            service.submit(job);
        }
        service.shutdown();
    }
}
```

Output:

```
Ram ...Job started by Thread: pool-1-thread-3
Parvati ...Job started by Thread: pool-1-thread-2
Shiv ...Job started by Thread: pool-1-thread-1
Ram ...Job completed by Thread: pool-1-thread-3
Parvati ...Job completed by Thread: pool-1-thread-2
Shiv ...Job completed by Thread: pool-1-thread-1
Sita ...Job started by Thread: pool-1-thread-2
Krishna ...Job started by Thread: pool-1-thread-3
Krishna ...Job completed by Thread: pool-1-thread-3
Sita ...Job completed by Thread: pool-1-thread-2
```

Why Use Thread Pool?

Without Thread Pool:

- High memory usage
- Slow performance(thread creation is expensive)
- Difficult to manage Threads

With thread pool:

- Reuses thread
- Improves performance
- Controls number of threads
- Better resource management

Executor Framework

Java provides the Executor framework (java.util.concurrent) to manage thread pools.

Main interface/Classes:

- Executor
- ExecutorService
- ScheduledExecutorService
- Executors(utility class)

Types of Thread Pool

1. Fixed Thread Pool

```
    ExecutorService executor = Executors.newFixedThreadPool(3);
```

2. **Cached Thread Pool**

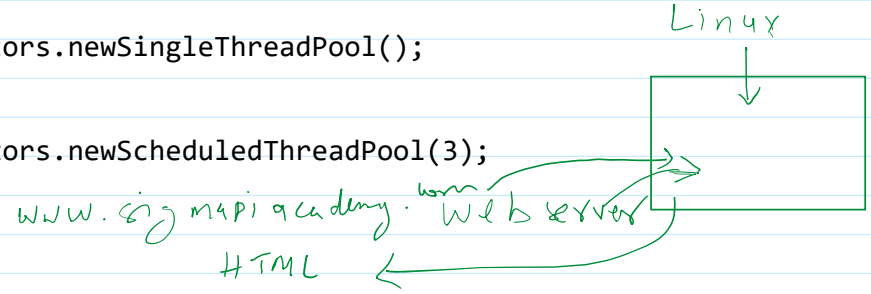
```
    ExecutorService executor = Executors.newCachedThreadPool();
```

3. **Single Thread Executor**

```
    ExecutorService executor = Executors.newSingleThreadPool();
```

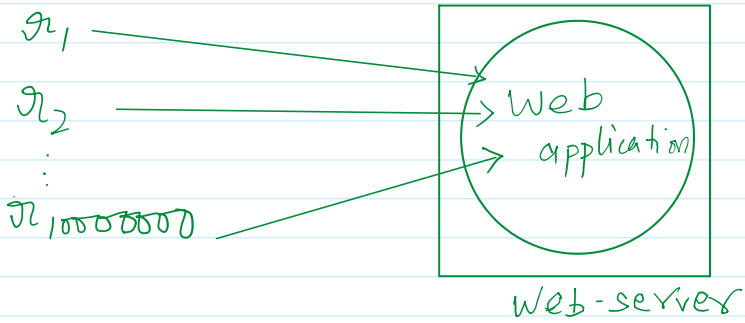
4. **Scheduled Thread Pool**

```
    ExecutorService executor = Executors.newScheduledThreadPool(3);
```



How it works?

1. **Create a Thread Pool**
2. **Submit tasks**
3. **Threads execute tasks**
4. **Reuse threads**
5. **Shutdown pool**



What is the need of Thread Pool?

While designing web-server or application-server, we can use Thread Pool concept.

Callable Interface

In Java, the Callable interface is used in Multithreading when you want a task to:

- Return a result
- Throw checked exceptions

Callable interface is a part of java.util.concurrent package
It is similar to Runnable interface, but with two important differences:

Feature	Runnable	Callable
Return value	**No**	**Yes**
Throws Exception	**No**	**Yes**
Method name	**run()**	**call()**

```
class MyCallable implements Callable{
  …
  …
  public Object call() throws Exception{
     …
     …
  }
}
```

```
class MainClass{
  public static void main(String[] args){
     MyCallable C = {,,,,,,,,};
     ExecutorService service = Executors.newFixedThread(3);
     for(Callable c1 : C){
        Future result = services.submit(c1);
        Sout(result.get())
     }

  }
}
```

Official docs:
Thread Pools
Most of the executor implementations in java.util.concurrent use *thread pools*, which consist of *worker threads*. This kind of thread exists separately from the Runnable and Callable tasks it executes and is often used to execute multiple tasks.
Using worker threads minimizes the overhead due to thread creation. Thread objects use a significant amount of memory, and in a large-scale application, allocating and deallocating many thread objects creates a significant memory management overhead.
One common type of thread pool is the *fixed thread pool*. This type of pool always has a specified number of threads running; if a thread is somehow terminated while it is still in use, it is automatically replaced with a new thread. Tasks are submitted to the pool via an internal queue, which holds extra tasks whenever there are more active tasks than threads.
An important advantage of the fixed thread pool is that applications using it *degrade gracefully*. To understand this, consider a web server application where each HTTP request is handled by a separate thread. If the application simply creates a new thread for every new HTTP request, and the system receives more requests than it can handle immediately, the application will suddenly stop responding to *all* requests when the overhead of all those threads exceed the capacity of the system. With a limit on the number of the threads that can be

created, the application will not be servicing HTTP requests as quickly as they come in, but it will be servicing them as quickly as the system can sustain.
A simple way to create an executor that uses a fixed thread pool is to invoke the newFixedThreadPool factory method in java.util.concurrent.Executors This class also provides the following factory methods:

- The newCachedThreadPool method creates an executor with an expandable thread pool. This executor is suitable for applications that launch many short-lived tasks.
- The newSingleThreadExecutor method creates an executor that executes a single task at a time.
- Several factory methods are ScheduledExecutorService versions of the above executors.

If none of the executors provided by the above factory methods meet your needs, constructing instances of java.util.concurrent.ThreadPoolExecutor or java.util.concurrent.ScheduledThreadPoolExecutor will give you additional options.